



PRESENTASI PROYEK AKHIR

Basis Data Terdistribusi

Replikasi Master-Slave • Docker • Laravel
Filament

IMPLEMENTASI SISTEM BASIS DATA TERDISTRIBUSI DENGAN REPLIKASI MASTER- SLAVE DAN DASHBOARD MONITORING BERBASIS WEB UNTUK JURNAL TRADING

Studi Kasus: **Aplikasi Jurnal Trading Berbasis Web**

Deby Hendri Yarto

NIM: 20220801294

Program Studi Teknik Informatika

Fakultas Ilmu Komputer — Universitas Esa Unggul

Mata Kuliah: Sistem Basis Data Terdistribusi (SBDT) — Semester Genap 2025/2026

Latar Belakang Masalah

- 1 Aktivitas jurnal trading (entry, hasil win/loss, nominal profit/loss) dicatat harian dan menghasilkan volume data transaksi yang terus bertambah setiap saat.
- 2 Jika seluruh proses input data dan pembacaan dashboard analitik dibebankan pada satu server database yang sama, risiko gangguan (downtime) saat server sibuk menjadi tinggi.
- 3 Kebutuhan akan ketersediaan tinggi (high availability): jika server utama gagal, pencatatan transaksi tidak boleh berhenti dan data historis tidak boleh hilang.
- 4 Belum ada pemisahan yang jelas antara server untuk transaksi (write) dan server untuk pelaporan/monitoring (read), sehingga beban query analitik dapat mengganggu performa input data.

Rumusan Masalah dan Tujuan Penelitian

RUMUSAN MASALAH

- Bagaimana mekanisme replikasi Master-Slave dapat menjamin konsistensi data jurnal trading antara server utama dan server cadangan?
- Bagaimana memisahkan beban kerja input transaksi (write) pada Master dan beban kerja dashboard monitoring (read) pada Slave?
- Bagaimana Docker dapat digunakan untuk menyediakan lingkungan basis data terdistribusi yang mudah direplikasi di server berbeda?

TUJUAN PENELITIAN

- Membangun sistem basis data terdistribusi dengan replikasi Master-Slave asynchronous untuk aplikasi Jurnal Trading.
- Mengimplementasikan seluruh infrastruktur (Master, Slave, aplikasi web) dalam container Docker agar mudah direproduksi.
- Menyediakan dashboard monitoring berbasis web (Laravel Filament) yang membaca data dari server Slave.

Batasan Masalah dan Ruang Lingkup

Termasuk Ruang Lingkup

- Replikasi MySQL Master-Slave (asynchronous) dengan 1 Master dan 1 Slave
- Containerization seluruh service menggunakan Docker & Docker Compose
- Aplikasi input data (CRUD) jurnal trading berbasis Laravel Filament
- Dashboard monitoring performa trading yang membaca dari Slave

Di Luar Ruang Lingkup

- Skema multi-master atau cluster replikasi (Galera, NDB Cluster)
- Load balancer / reverse proxy tingkat lanjut (Nginx HA, Keepalived)
- Manajemen risiko trading atau strategi trading itu sendiri
- Pengujian pada skala production dengan jumlah pengguna besar

Studi Literatur

Judul dan Sumber	Tahun	Hasil Utama	Relevansi
Sholihah, I. — Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker (Majalah Ilmiah Teknologi Elektro)	2022	Replikasi Master-Slave pada MySQL berbasis Docker terbukti menyalin data secara konsisten antara server master dan slave.	Justifikasi teknik replikasi & Docker
Penerapan Disaster Recovery Plan Pada Database MySQL Menggunakan Metode Master to Master Replication Berbasis Docker (Jurnal Komputer Terapan)	2024	Replikasi database berbasis Docker efektif sebagai strategi pemulihan bencana (disaster recovery) data.	Justifikasi ketersediaan data
Pamungkas, et al. — Performance Evaluation of a Laravel-based Internship Website using MySQL Master–Slave Replication (Sistemasi: Jurnal Sistem Informasi)	2026	Konfigurasi Master-Slave pada aplikasi Laravel ber-container memberikan performa lebih stabil dibanding single database saat beban meningkat.	Justifikasi arsitektur Laravel + Master-Slave
High Availability and Performance of Database in the Cloud — Traditional Master-Slave Replication versus Modern Cluster-based Solutions	2017	Replikasi Master-Slave tetap relevan untuk high availability, dibandingkan dengan solusi cluster modern seperti Galera.	Perbandingan pendekatan replikasi

Landasan Teori

Replikasi Basis Data

Proses menyalin dan mensinkronkan data dari satu server database (Master) ke satu atau lebih server lain (Slave) secara otomatis, guna menjaga ketersediaan dan redundansi data.

Asynchronous vs Synchronous

Replikasi asynchronous mengirim perubahan data ke Slave tanpa menunggu konfirmasi, menghasilkan performa lebih baik untuk aplikasi web dibanding replikasi synchronous yang menunggu Slave selesai menulis.

Docker & Containerization

Docker Compose menyediakan lingkungan terisolasi (isolated environment) untuk setiap service (Master, Slave, aplikasi), mencegah konflik dependensi saat dijalankan di server berbeda.

Laravel & Filament

Laravel sebagai framework backend, Filament sebagai panel admin berbasis web untuk input data jurnal trading dan menampilkan dashboard monitoring secara reaktif.

Metodologi Penelitian

Metode Pengembangan Sistem: Prototyping — sistem dibangun bertahap (Master, lalu Slave, lalu Dashboard) dengan pengujian pada tiap tahap sebelum lanjut ke tahap berikutnya.

01

Analisis Kebutuhan

Menentukan kebutuhan fungsional: input transaksi, dashboard monitoring, dan kebutuhan non-fungsional: ketersediaan data.

02

Perancangan Sistem

Merancang arsitektur replikasi, skema tabel trades, serta struktur docker-compose.yml untuk tiap service.

03

Implementasi

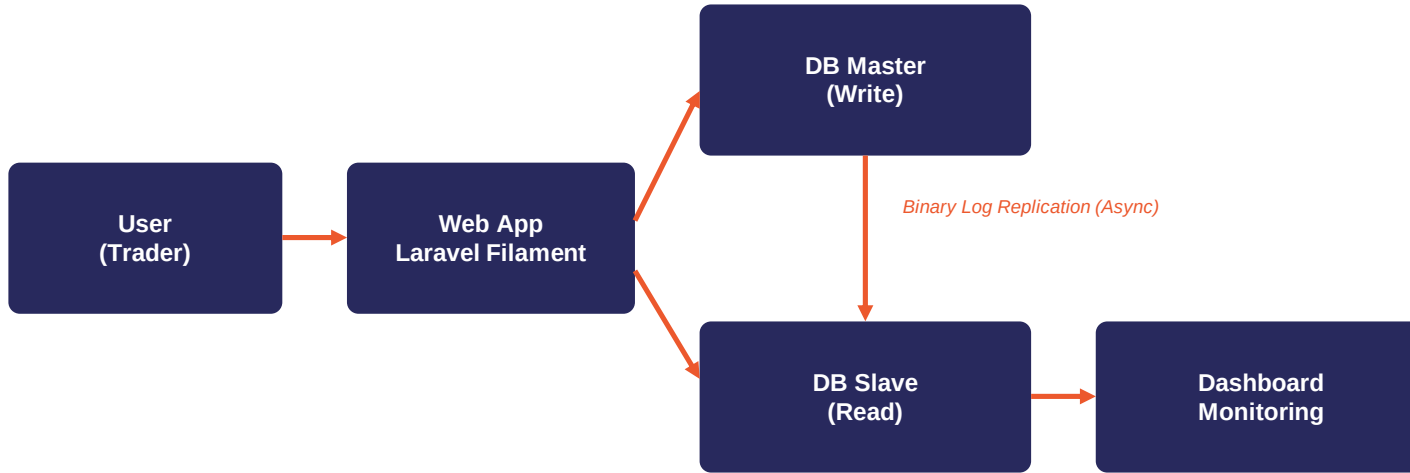
Konfigurasi Master & Slave (my.cnf), menjalankan container, serta membangun aplikasi Filament di atas Laravel.

04

Pengujian & Analisis

Menguji konsistensi data antar server serta menganalisis performa dashboard yang mengakses Slave.

Arsitektur Sistem



Alur: User berinteraksi dengan aplikasi Laravel Filament. Semua transaksi tulis (input jurnal trading) diarahkan ke DB Master. Perubahan pada Master direplikasi secara asynchronous ke DB Slave melalui binary log. Dashboard monitoring membaca data khusus dari DB Slave, sehingga beban query analitik tidak mengganggu performa transaksi pada Master.

Perancangan Basis Data

Tabel utama trades — direplikasi dari Master ke Slave

Kolom	Tipe Data	Keterangan
id	BIGINT (PK, auto increment)	Identitas unik tiap catatan transaksi trading
tanggal	DATE	Tanggal transaksi dilakukan
pair	VARCHAR(20)	Pasangan instrumen yang ditradingkan
hasil_wl	ENUM('win','loss')	Hasil transaksi: Win atau Loss
jumlah_rupiah	DECIMAL(15,2)	Nominal profit/loss dalam Rupiah
created_at	TIMESTAMP	Waktu data dicatat, dipakai untuk audit replikasi

Prinsip Replikasi Skema

- Struktur tabel dibuat identik di Master dan Slave sebelum replikasi diaktifkan.
- Hanya Master yang menerima perintah INSERT/UPDATE/DELETE.
- Slave bersifat read-only dan menerima perubahan lewat binary log Master.
- Konsistensi diverifikasi dengan membandingkan jumlah baris pada kedua server.

Implementasi: Konfigurasi Master dan Slave

my.cnf – MASTER

```
[mysqld]
server-id=1
log_bin=/var/log/mysql/mysql-bin.log
binlog_do_db=jurnal_trading
```

Perintah Replikasi (di Master)

```
CREATE USER 'replica'@'%'
  IDENTIFIED BY 'password';
GRANT REPLICATION SLAVE
  ON *.* TO 'replica'@'%;
FLUSH PRIVILEGES;
SHOW MASTER STATUS;
```

my.cnf – SLAVE

```
[mysqld]
server-id=2
relay-log=/var/log/mysql/mysql-relay-bin
```

Perintah Replikasi (di Slave)

```
CHANGE MASTER TO
  MASTER_HOST='master_ip',
  MASTER_USER='replica',
  MASTER_PASSWORD='password',
  MASTER_LOG_FILE='...',
  MASTER_LOG_POS=...;
START SLAVE;
```

Implementasi: Verifikasi Replikasi dan Dashboard

Verifikasi Status Replikasi

```
docker exec -it db_master mysql -u root -p \  
-e "SHOW MASTER STATUS\G"  
  
docker exec -it db_slave mysql -u root -p \  
-e "SHOW SLAVE STATUS\G"
```

Indikator keberhasilan: Slave_IO_Running: Yes dan Slave_SQL_Running: Yes pada output SHOW SLAVE STATUS menandakan Slave berhasil menarik dan menerapkan data dari binary log Master secara otomatis.

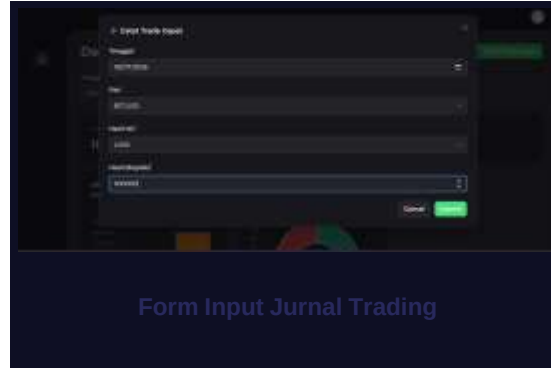
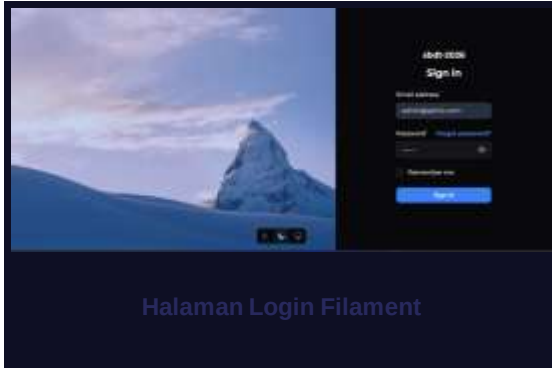
Dashboard Monitoring

Panel admin Filament (Master): digunakan untuk input dan pengelolaan data jurnal trading secara langsung.

Panel dashboard (Slave): menampilkan rekap dan filter data historis (contoh: filter tanggal transaksi) tanpa membebani server Master.

Cara membedakan Master/Slave: dilihat dari nilai server-id pada docker-compose.yml — server-id: 1 adalah Master, server-id: 2 adalah Slave.

Tampilan Aplikasi (Bukti Implementasi)



Alur pengujian: Admin login ke panel Filament, mencatat transaksi baru lewat form "Catat Trade Cepat" yang tersimpan ke DB Master. Data tersebut kemudian direplikasi ke DB Slave, dan langsung terlihat di Dashboard Monitoring — dibuktikan dengan bertambahnya Total Trades dan berubahnya grafik Win Rate serta Net P/L secara otomatis setelah data baru dicatat.

Pengujian dan Analisis Hasil

Skenario Pengujian	Langkah	Hasil	Status
Konsistensi data	100x INSERT pada Master, lalu SELECT count(*) FROM trades di Slave	Jumlah baris pada Slave identik dengan Master	Valid
Status replikasi	SHOW SLAVE STATUS pada container Slave	Slave_IO_Running dan Slave_SQL_Running = Yes	Valid
Ketahanan saat Master mati	Menghentikan container Master sementara	Data pada Slave tetap dapat diakses dashboard monitoring	Valid
Beban CPU dashboard	Mengakses dashboard langsung ke Slave vs ke Master	Beban CPU pada Master berkurang karena query analitik dialihkan ke Slave	Tercapai

Analisis: Pengujian membuktikan bahwa replikasi asynchronous berhasil menjaga konsistensi data tanpa membebani proses transaksi pada Master. Pemisahan beban baca (dashboard) ke Slave terbukti mengurangi beban komputasi pada server utama, sesuai dengan tujuan penelitian.

Kesimpulan dan Saran

KESIMPULAN

- Sistem replikasi Master-Slave berhasil diimplementasikan menggunakan Docker dan menjaga konsistensi data jurnal trading secara otomatis.
- Pemisahan beban antara Master (write) dan Slave (read) terbukti mengurangi beban komputasi pada server utama.
- Dashboard monitoring berbasis Laravel Filament berhasil menampilkan data historis jurnal trading langsung dari Slave.

SARAN

- Menambahkan skema multi-slave atau load balancer untuk skala pengguna yang lebih besar.
- Menerapkan mekanisme automatic failover jika Master mengalami gangguan.
- Menambahkan monitoring lag replikasi secara real-time pada dashboard.

Daftar Pustaka

- Sholihah, I. (2022). Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker. Majalah Ilmiah Teknologi Elektro, 21(2). <https://doi.org/10.24843/MITE.2022.v21i02.P08>
- Penerapan Disaster Recovery Plan Pada Database MySQL Menggunakan Metode Master to Master Replication Berbasis Docker. (2024). Jurnal Komputer Terapan, 10(1), 78–85.
- Pamungkas, et al. (2026). Performance Evaluation of a Laravel-based Internship Website using MySQL Master–Slave Replication. Sistemasi: Jurnal Sistem Informasi.
- High Availability and Performance of Database in the Cloud — Traditional Master-slave Replication versus Modern Cluster-based Solutions. (2017).
- Kane, S. P., & Matthias, K. (2023). Docker: Up & Running: Shipping Reliable Containers in Production. O'Reilly Media.